

ТЗшка

Вот готовое техническое задание (ТЗ) и описание проекта, которое бьёт ровно в требования вашей методички, не содержит лишней работы и чётко описывает реализацию идеи «Prompt Improver». [PMVS-Proekt-2025-2026.pdf](#)

Техническое задание: Проект «PromptForge» (Prompt Improver)

1. Концепция и описание приложения

PromptForge — это Android-приложение для автоматического улучшения пользовательских промптов к нейросетям. Пользователь вводит черновой запрос, приложение через Google Gemini API задаёт 2-3 уточняющих вопроса в виде кнопок (1 клик = 1 ответ), а затем генерирует идеальный промпт. Приложение сохраняет историю запросов локально и обучается на фидбеке пользователя, подстраивая системный промпт под его стиль.

2. Покрытие требований лабораторной работы (Чек-лист)

Проект спроектирован так, чтобы закрыть 100% требований методички без "оверинжиниринга": [PMVS-Proekt-2025-2026.pdf](#)

- **Многооконный интерфейс & Jetpack Compose:** Приложение состоит из 3 экранов (список, генератор, детали), реализованных на Jetpack Compose с навигацией (Navigation Compose).
- **Хранение данных (Room):** Сохранение истории промптов и пользовательских предпочтений в локальную SQLite БД через библиотеку Room.
- **Внешний API & Многопоточность:** Асинхронные сетевые вызовы к Google Gemini API 1.5 Flash через Kotlin Coroutines (Dispatchers.IO), чтобы не блокировать UI-поток.
- **Тач-интерфейс:** Использование свайпов (удаление промпта из истории), кликабельных чипсов (ответы на вопросы ИИ), системы оценки (звёзды/лайки).
- **Командная работа & CI/CD:** GitHub Projects для задач, GitHub Actions для автоматической сборки APK и прогона Unit/UI тестов (в т.ч. Firebase Test Lab), Wiki и README.

3. Архитектура экранов (Минимум для сдачи)

Экран 1: Главный (История)

- Список всех ранее сгенерированных промптов (подтягивается из Room).
- Элементы тач-интерфейса: свайп влево для удаления, клик для перехода к деталям.
- FAB (Floating Action Button) для создания нового промпта.

Экран 2: Интерактивный генератор (Improver Flow)

- *Шаг 1:* Текстовое поле ввода ("Напиши короткий промпт"). Кнопка "Улучшить".
- *Шаг 2 (Сетевой запрос):* Показ индикатора загрузки. Появление 2-3 вопросов от LLM. Варианты ответов выводятся в виде кликабельных кнопок-чипсов.
- *Шаг 3 (Сетевой запрос):* После кликов по ответам генерируется финальный промпт. Кнопка "Скопировать".
- *Шаг 4 (Оценка):* Блок фидбека: Оценка (1-5), выбор причины ("Слишком длинно", "Не тот формат"), текстовый коммент. Кнопка "Сохранить".

Экран 3: Детали промпта

- Просмотр старого промпта из истории: исходный текст, улучшенный текст, дата, оставленный фидбек.

4. Схема базы данных (Room)

Достаточно двух таблиц (Entity), чтобы показать связь и работу с БД.

1. Таблица Prompts (История)

- `id` (Primary Key, Int)
- `raw_prompt` (String) — исходный текст.
- `improved_prompt` (String) — результат.
- `rating` (Int) — оценка пользователя.
- `timestamp` (Long) — дата создания.

2. Таблица UserPreferences (Профиль обучения)

- `id` (Primary Key = 1, всегда одна запись).
- `custom_instructions` (String) — текущие инструкции для ИИ (например: "Пользователь не любит длинные промпты, предпочитает формат Markdown"). Обновляется после каждого фидбека.

5. Взаимодействие с API (Google Gemini 1.5 Flash)

Реализуется через бесплатный ключ Google AI Studio. Используются 3 типа запросов (многопоточность через Coroutines):

- **API Call 1 (Генерация вопросов):**
 - *Запрос:* Исходный промпт + `custom_instructions` из БД. Системный промпт просит вернуть строго JSON: `[{"question": "Формат?", "options": ["Текст", "Таблица", "Код"]}].`
 - *Ответ:* Android парсит JSON и рисует кнопки.
- **API Call 2 (Финальный промпт):**
 - *Запрос:* Исходный промпт + выбранные ответы.
 - *Ответ:* Текст идеального промпта.
- **API Call 3 (Уточнение предпочтений):**
 - *Запрос:* Отправляется в фоне после того, как юзер дал фидбек. Передаются старые `custom_instructions`, сам промпт и комментарий юзера ("сделай короче"). Gemini возвращает обновленный текст `custom_instructions`, который перезаписывается в Room.

6. Требования к документированию и репозиторию [PMVS-Proekt-2025-2026.pdf](#)

Чтобы преподаватель принял проект, репозиторий должен быть настроен строго по методичке:

- **Организация в GitHub:** Создать Organization, добавить туда 2-3 участников.
- **Два репозитория (или подмодули):** `PromptForge-App` (код) и `PromptForge-Docs` (документация). Репозиторий с кодом добавляется в репозиторий с документацией как *git submodule*.
- **Readme.md:** Блоки Project Name, Description, Installation, Usage (со скриншотами), Contributing (кто что делал).
- **GitHub Pages / Wiki:**
 - *Главная страница.*
 - *Функциональные требования:* Диаграмма Use Case (создание промпта, оценка, просмотр истории) и текстовые сценарии.
 - *Схема БД:* Картинка со связями `Prompts` и `UserPreferences` + ссылка на SQL-файл.
 - *Архитектура файлов:* Описание слоёв (UI, ViewModel, API, Room).

- **GitHub Actions:** Настроенный CI pipeline (`.yaml` файл), который при пуше в `main` запускает Unit-тесты и собирает `.apk` файл. Для UI-тестов добавляется интеграция с Firebase Test Lab (потребуется написать 1 базовый UI-тест на Compose, например, проверку появления списка).